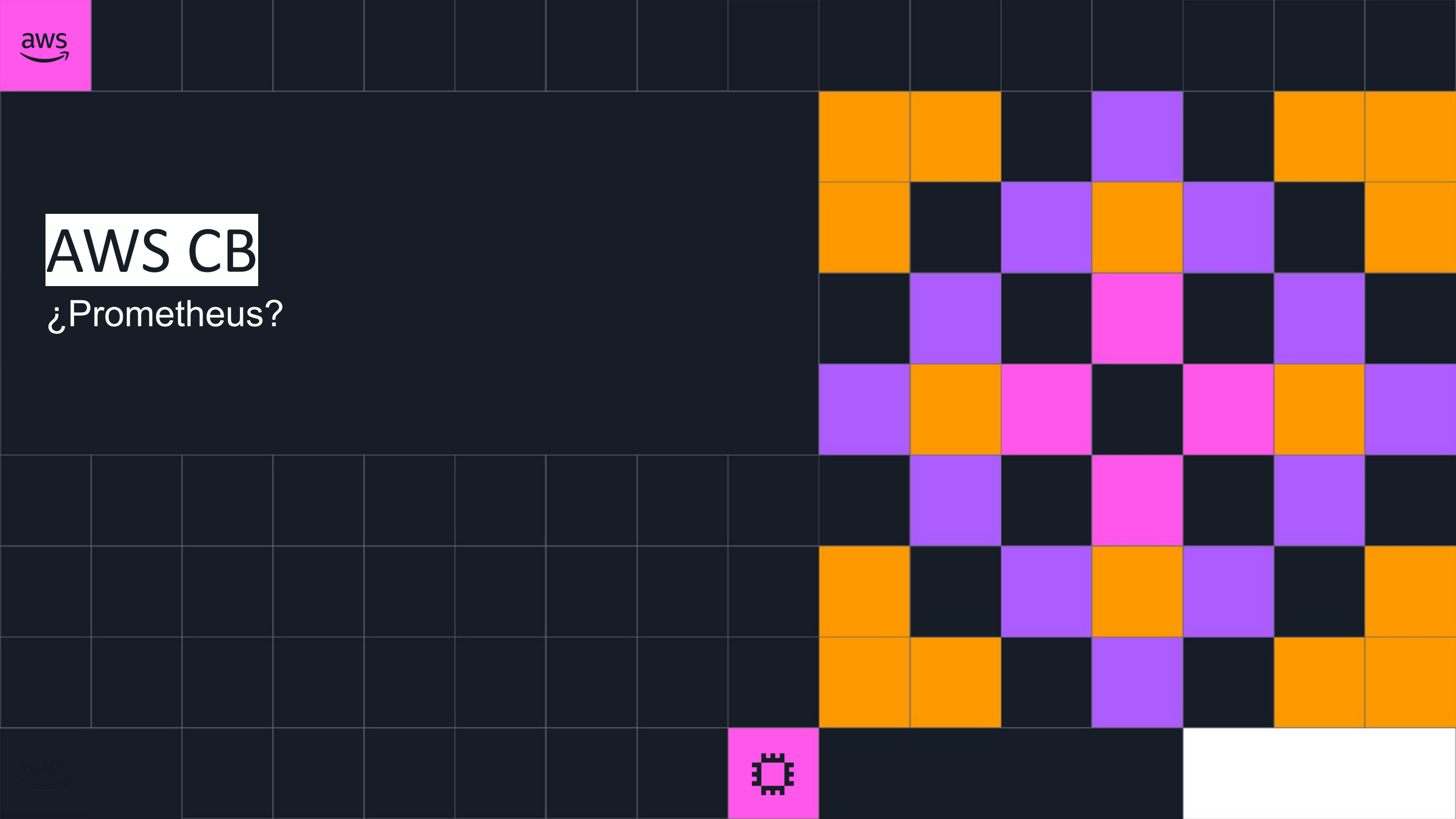

aws
AWS CB
¿Prometheus?
aws

Jhonnatan Gil / JhonnyPong

DevOps / SRE / observability

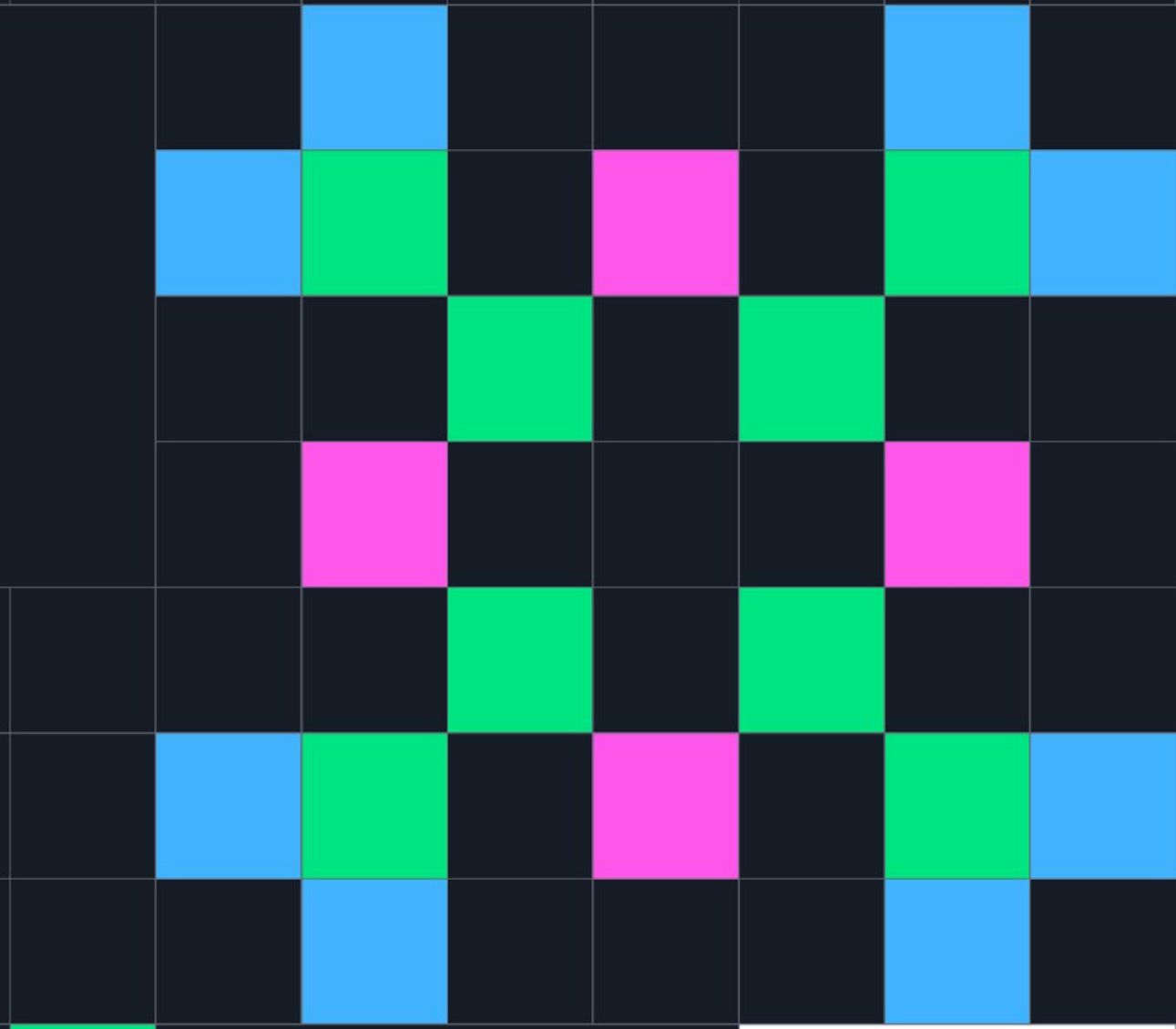
Globant

CB - Contenedores



/in/jhonnypong/

LEVEL 1: TELEMETRÍA UNIFICADA & AMP



Dominar la telemetría unificada antes del Game Over operacional

01

Las 3 Señales

02

Enfréntate al Jefe

03

Power-Up AWS AMP

04

Las 3 Señales

Diferenciar Métricas, Logs y Trazas como HP,

Event History y Quest Maps.



METRICS = HP / STATUS

Valores numéricos agregados en series de tiempo. Indican la salud general del sistema inmediatamente.

LOGS = EVENT HISTORY

Registros de texto con timestamp de eventos discretos. Explican exactamente qué ocurrió en fallas.

TRACES = QUEST MAP

Ruta de una solicitud a través de microservicios. Revelan los cuellos de botella en la aventura.

Enfréntate al Jefe

Analizar la arquitectura de Prometheus Self-Hosted y su carga operativa.



RADAR: **INFRA** VS. APPS

METRICAS DE INFRAESTRUCTURA

Capa de Servidores & Contenedores

CPU Usage: Uso de procesamiento del host.

RAM Utilization: Consumo de memoria física.

Network I/O: Ancho de banda entrante/saliente.

RADAR: INFRA VS. **APPS**

METRICAS DE APLICACIÓN

Capa de Código & Negocio

Request Rate: Solicitudes procesadas por segundo.

Error Rate: Porcentaje de HTTP 5xx/4xx.

Latency: Duración de respuestas en p95/p99.

PROMETHEUS CLASSIC ARCHITECTURE

01

Pull/Scrape Model: Prometheus consulta periódicamente los endpoints HTTP `/metrics` de los objetivos registrados en su configuración.

02

TSDB Local: Almacena las métricas en un Time Series Database local eficiente, optimizado para rápida escritura de datos temporales.

03

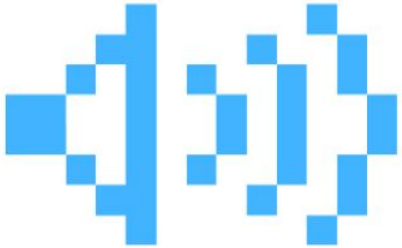
PromQL: Lenguaje de consultas funcional para calcular tasas, percentiles y agrupar dimensiones en tiempo real.

04

OPERATIONAL BURDEN

DISCOS TSDB LLENOS

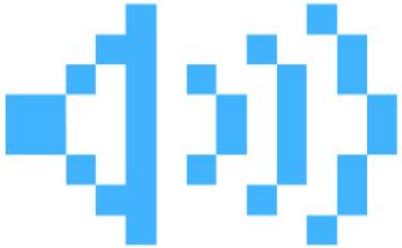
La alta cardinalidad agota rápidamente el almacenamiento local requerimiento gestión manual de retención.



OPERATIONAL BURDEN

HA COMPLEJA

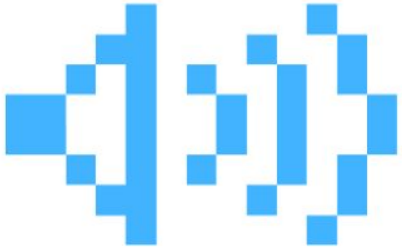
Prometheus no escala horizontalmente por defecto; requiere Thanos o Cortex para alta disponibilidad.



OPERATIONAL BURDEN

MANTENIMIENTO CONTINUO

Actualización constante de parches de seguridad, backups y capacidad de cómputo dedicada.



Power-Up AWS AMP

Escalar métricas con AWS Managed Service for Prometheus e IAM SignV4.



AWS MANAGED PROMETHEUS (AMP)

Serverless Monitoring: Elimina la necesidad de aprovisionar, escalar o mantener infraestructura de almacenamiento TSDB.

Autoscale Automático: Escala ingesta, almacenamiento y ejecución de consultas PromQL dinámicamente según la carga.

Seguridad Integrada: Control de acceso mediante AWS IAM SigV4 y cifrado nativo en reposo y en tránsito.

SELF-HOSTED VS. AWS AMP

CARACTERÍSTICA	PROMETHEUS SELF-HOSTED	AWS MANAGED PROMETHEUS (AMP)
Mantenimiento	Alto (Parches, discos, paradas)	Zero (Managed Service)
Escalabilidad	Manual (Requiere Thanos/Cortex)	Autoscaling Automático
Seguridad	Configuración manual de TLS/Auth	Nativa con AWS IAM SigV4
Modelo de Costos	Instancias fijas + EBS 24/7	Pago por ingesta de métricas y consultas

Hands On - INSTRUMENTANDO PYTHON



OBJETIVO DEL TALLER

Exponer métricas de la aplicación en formato texto estándar de Prometheus a través del endpoint `/metrics`.

LIBRERÍAS CLAVE

Uso de `flask` para la app web y `prometheus_client` para la generación e incremento de métricas.

```python
from flask import Flask
from prometheus_client import Counter, generate_latest

app = Flask(__name__)
# Definición de la métrica Counter
REQUEST_COUNT = Counter('http_requests_total', 'Total HTTP Requests', ['method', 'endpoint'])

@app.route('/')
def home():
    REQUEST_COUNT.labels(method='GET', endpoint='/').inc()
    return "Player 1 Ready!"

@app.route('/metrics')
def metrics():
    return generate_latest(), 200, {'Content-Type': 'text/plain'}

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

https://github.com/jthan24/ucsg-reposotories/tree/main/01-dia



MUCHAS GRACIAS

Jhonnatan Gil / JhonnyPong

DevOps / SRE / observability

Globant

CB - Contenedores

/in/jhonnypong/

